# 6 Support Vector Machines

The essence of Support Vector Machines (SVMs) is best illustrated visually. Figure 6.1 displays a segment of the iris dataset discussed previously. This dataset is linearly separable as the two classes can be distinctly divided with a straight line. The left plot reveals the decision boundaries for three different linear classifiers. One model, depicted by the dashed line, fails to separate the classes adequately. The other two models accurately classify the training data; however, their decision boundaries are very close to the data points, which may lead to poorer performance on new data. In contrast, the solid line in the right plot delineates the SVM classifier's decision boundary, which not only separates the classes effectively but also maximizes the distance from the nearest data points. This approach is known as large margin classification, where the SVM strives to fit the broadest possible "street" (indicated by the parallel dashed lines) between the classes.

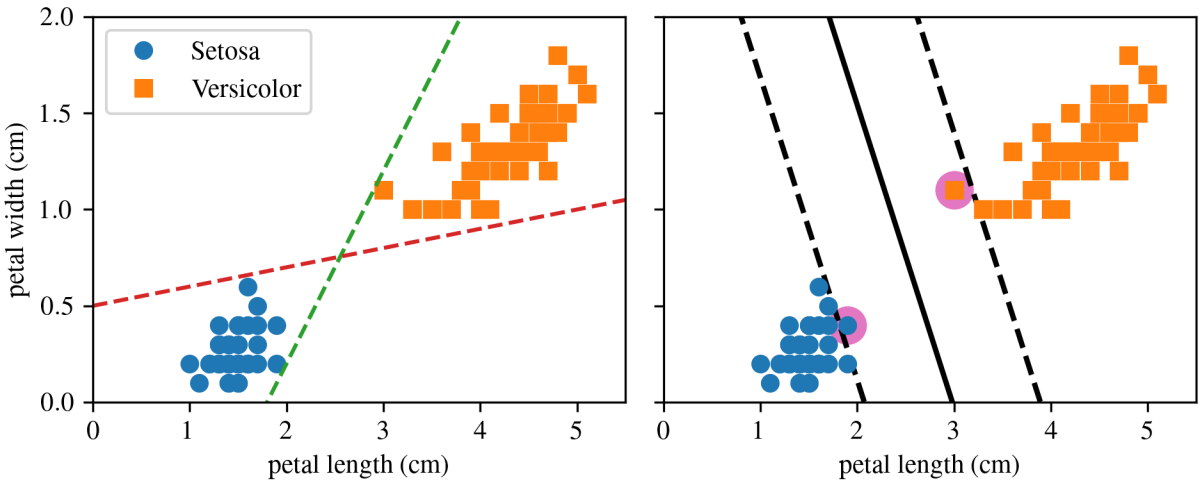


Figure 6.1: Large margin classification.

Observe that the addition of further training instances outside the "street" does not influence the decision boundary; it is entirely shaped by the instances situated on the boundary's edge. These pivotal instances are termed support vectors and are highlighted with circles in Figure 6.1.

NOTE

The sensitivity of SVMs to feature scales is evident in Figure 6.2. In the left plot, the vertical dimension greatly outweighs the horizontal dimension, resulting in a nearly horizontal "street." However, after applying feature scaling such as using Scikit-Learn's `StandardScaler` the decision boundary becomes more appropriate, as illustrated in the right plot.

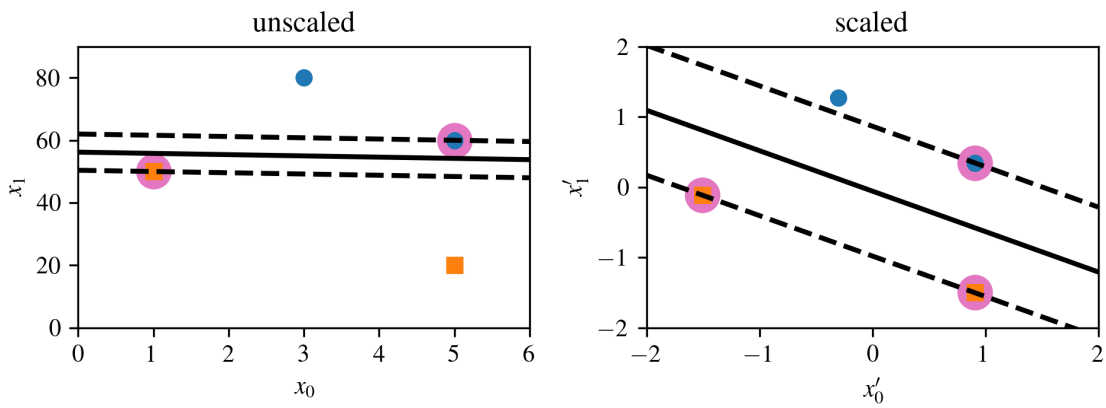


Figure 6.2: Sensitivity to feature scales.

## 6.1 Linear SVM Classification

Hard margin classification demands that all instances be correctly classified without any margin violations. This strict approach faces two significant challenges:

- It is only feasible when the data is linearly separable.
- It is highly sensitive to outliers.

Figure 6.3 illustrates these challenges using the iris dataset with an added outlier. On the left, achieving a hard margin is impossible due to the outlier. On the right, although a decision boundary is found, it deviates substantially from the optimal boundary shown in Figure 6.1 and is less likely to perform well on new data.

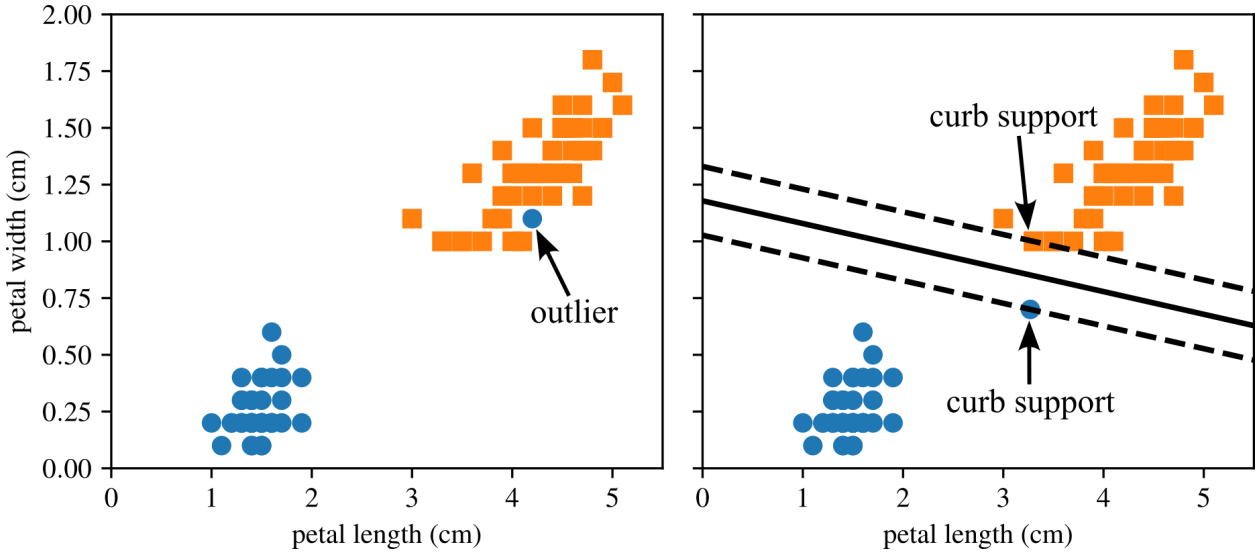


Figure 6.3: Support vector machines showing (left) an un-separable case, and (right) a separable case with two data points supporting the curbs of the support vector machine.

To mitigate the limitations of hard margin classification, a more adaptable model, known as soft margin classification, is often employed. The goal here is to achieve an optimal balance between maximizing the margin width and minimizing margin violations, where instances might fall into the margin or on the incorrect side.

Scikit-Learn's SVM implementations facilitate this balance through the hyperparameter $C$. A smaller value of $C$ results in a wider margin but allows more margin violations, which is beneficial for model flexibility. Conversely, a larger $C$ value tightens the margin, reducing margin violations but at the risk of a less flexible model. Figure 6.4 demonstrates this trade-off: the left plot with a low $C$ value

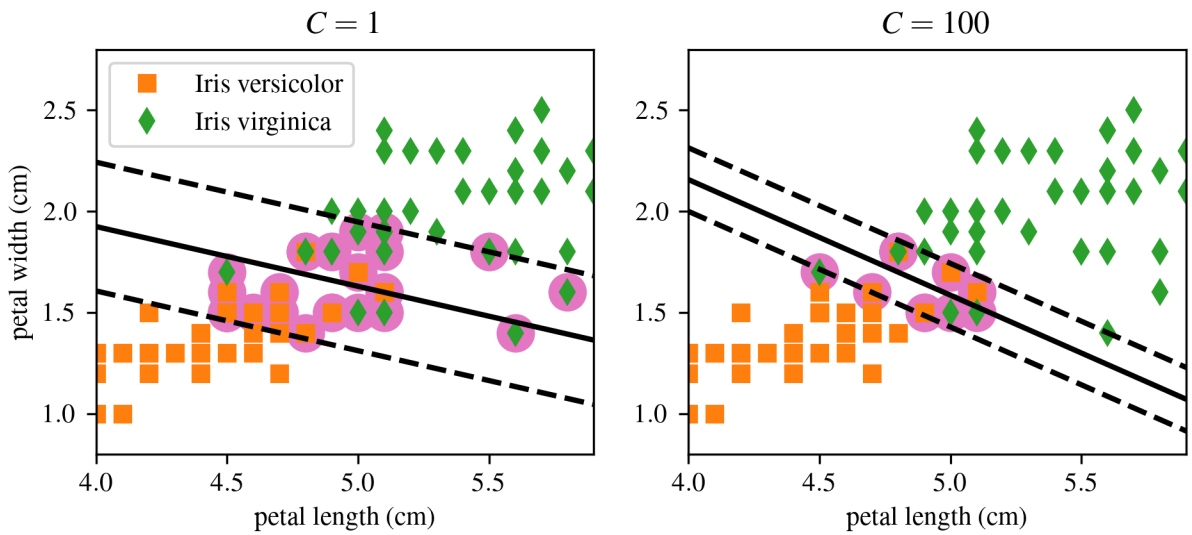


Figure 6.4: SVM margin sizes for different $C$ values.

**NOTE**

If your SVM model is overfitting, you can try regularizing it by reducing C.

In prior chapters, we adopted the convention of consolidating all model parameters into a single vector $\theta$, which included the bias term $\theta_0$ and the feature weights from $\theta_1$ to $\theta_n$, and we added a bias input $x_0 = 1$ to all instances. In this chapter, however, we adopt a different notation more suited to discussions on SVMs. Here, the bias term is denoted as $b$, and the vector of feature weights is denoted as $\mathbf{w}$. Importantly, no bias feature is added to the input feature vectors in this notation.

### 6.1.1 Decision Function and Predictions

A linear SVM classifier determines the class of a new instance $x$ by calculating the decision function $\mathbf{w}^{\mathrm{T}}\mathbf{x} + b = w_1 x_1 + \cdots + w_n x_n + b$. If the outcome is positive, the predicted class $\hat{y}$ is the positive class (1); otherwise, it is the negative class (0).

$$\hat{y} = \begin{cases} 0 & \text{if } \mathbf{w}^{\mathrm{T}}\mathbf{x} + b < 0, \\ 1 & \text{if } \mathbf{w}^{\mathrm{T}}\mathbf{x} + b \geq 0. \end{cases} \tag{6.6}$$

Figure 6.5 illustrates the decision function for a model; this is a two-dimensional plane given that the dataset has two features (petal width and petal length). The decision boundary, where the decision function equals zero, is depicted as a straight line (the thick solid line in the figure). The dashed lines indicate where the decision function equals 1 or -1 and are parallel to and equidistant from the decision boundary, effectively forming a margin around it. Training a linear SVM classifier involves optimizing the values of $\mathbf{w}$ and $b$ to maximize this margin while either avoiding margin violations (hard margin) or minimizing them (soft margin).

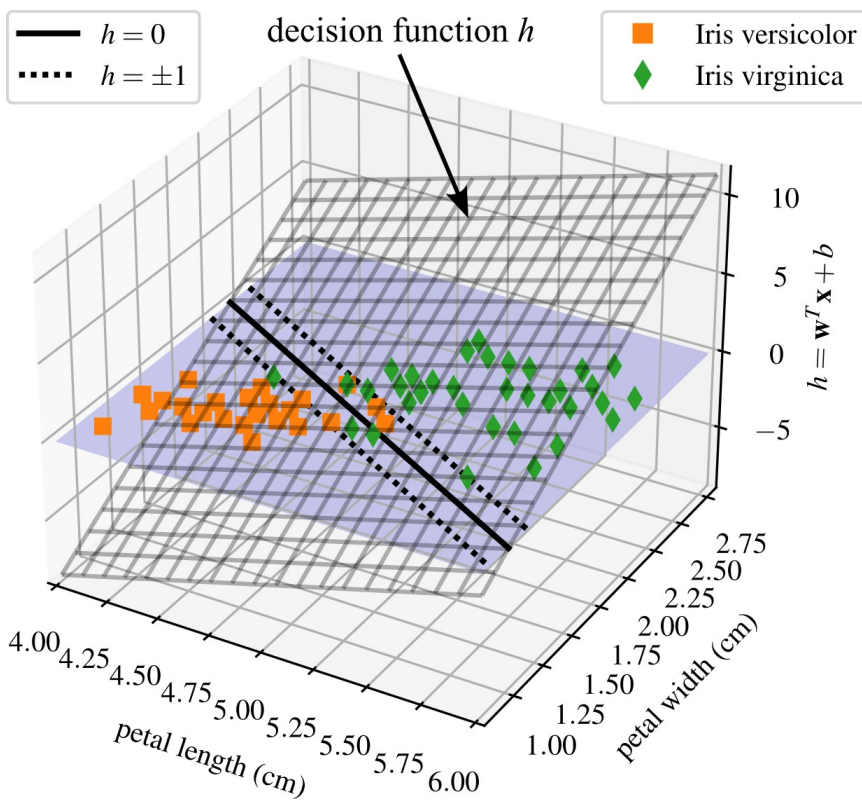


Figure 6.5: Decision function for the Iris Dataset showing how the decision function $h$ cuts through the feature space.

### 6.1.2 Training Objective

The slope of the decision function corresponds to the norm of the weight vector, $\|\mathbf{w}\|$. Halving this slope causes the decision boundary margins, where the decision function equals $\pm 1$, to double in distance from the decision boundary. Effectively, reducing the norm of $\mathbf{w}$ by half doubles the margin. This geometric interpretation is perhaps simpler to visualize in two dimensions, as shown in Figure 6.6. Thus, minimizing $\|\mathbf{w}\|$ maximizes the margin.

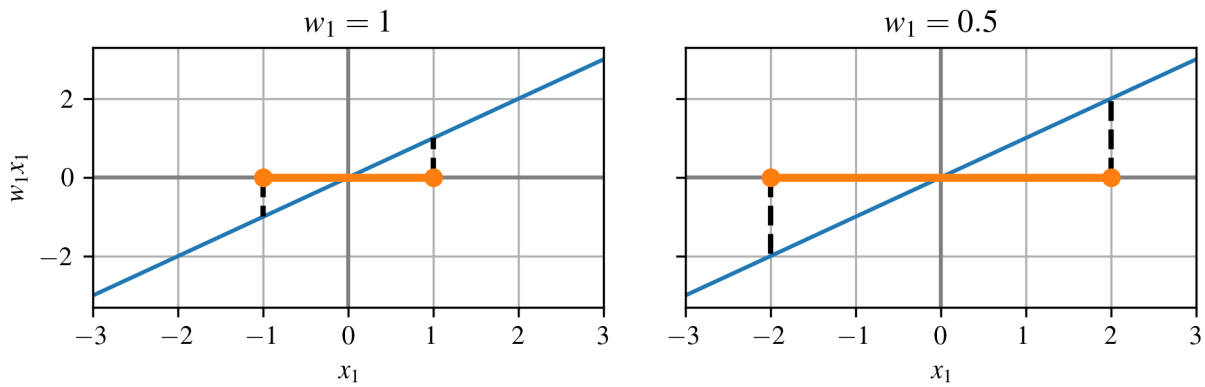


Figure 6.6: The margin is dependent on the value of the weight vector where a smaller weight vector results in a larger margin and vise versa.

To achieve a large margin while enforcing that no data points fall within the margin (hard margin), we ensure the decision function exceeds +1 for all positive training instances and is less than -1 for all negative instances. Let $t^{(i)}$ equal -1 for negative instances ($y^{(i)} = 0$) and +1 for positive ones ($y^{(i)} = 1$). The constraints then require $t^{(i)}(\mathbf{w}^{\mathrm{T}}\mathbf{x}^{(i)} + b) \geq 1$ for all training instances. This forms the basis of the hard margin linear SVM classifier optimization problem:

$$\begin{aligned} \underset{\mathbf{w},b}{\text{minimize}} \quad & \frac{1}{2}\mathbf{w}^{\mathrm{T}}\mathbf{w} \\ \text{subject to} \quad & t^{(i)}(\mathbf{w}^{\mathrm{T}}\mathbf{x}^{(i)} + b) \geq 1 \text{ for } i = 1, 2, \ldots, m \end{aligned} \tag{6.7}$$

**NOTE**

The objective function minimized is $\frac{1}{2}\mathbf{w}^{\mathrm{T}}\mathbf{w}$, equivalent to $\frac{1}{2}\|\mathbf{w}\|^2$. This formulation is chosen over minimizing $\|\mathbf{w}\|$ directly because $\frac{1}{2}\|\mathbf{w}\|^2$ offers a straightforward derivative, simply $\mathbf{w}$, facilitating gradient calculations. In contrast, $\|\mathbf{w}\|$ lacks differentiability at $\mathbf{w} = 0$, posing challenges for optimization algorithms, which typically require smooth, differentiable functions to ensure effective optimization.

To formulate the soft margin objective, it is necessary to introduce a slack variable $\zeta^{(i)} \geq 0$ for each instance. This variable, $\zeta^{(i)}$, quantifies the permissible margin violation for the i$^{\text{th}}$ instance. Consequently, we face dual objectives: minimizing the slack variables to reduce margin violations and minimizing $\frac{1}{2}\mathbf{w}^{\mathrm{T}}\mathbf{w}$ to maximize the margin. The hyperparameter $C$ plays a crucial role here, enabling a balance between these competing objectives. The introduction of $C$ transforms our task into a constrained optimization problem.

$$\begin{aligned} \underset{\mathbf{w},b,\zeta}{\text{minimize}} \quad & \frac{1}{2}\mathbf{w}^{\mathrm{T}}\mathbf{w} + C\sum_{i=1}^{m}\zeta^{(i)} \\ \text{subject to} \quad & t^{(i)}(\mathbf{w}^{\mathrm{T}}\mathbf{x}^{(i)} + b) \geq 1 - \zeta^{(i)} \text{ and } \zeta^{(i)} \geq 0 \text{ for } i = 1,\ 2,\ \cdots,\ m \end{aligned} \tag{6.8}$$

### 6.1.3 Quadratic Programming

Both hard margin and soft margin problems are examples of convex quadratic optimization problems with linear constraints, commonly referred to as Quadratic Programming (QP) problems. QP involves solving optimization problems where the objective is a quadratic function and the constraints are linear. This form of programming, established in the 1940s, predates and is distinct from "computer programming," and is sometimes more descriptively termed "quadratic optimization" to avoid confusion.

A variety of techniques available through off-the-shelf solvers can address these QP problems, though they extend beyond the scope of this text. The general formulation of a QP problem is as follows:

$$
\begin{aligned}
&\underset{\mathbf{p}}{\text{minimize}} \quad \frac{1}{2}\mathbf{p}^{\mathrm{T}}\mathbf{H}\mathbf{p} + \mathbf{f}^{\mathrm{T}}\mathbf{p} \\
&\text{subject to} \quad \mathbf{A}\mathbf{p} \leq \mathbf{b} \\
&\text{where} \quad \begin{cases} \mathbf{p} & \text{is an } n_p\text{-dimensional vector } (n_p = \text{number of parameters}), \\ \mathbf{H} & \text{is an } n_p \times n_p \text{ matrix}, \\ \mathbf{f} & \text{is an } n_p\text{-dimensional vector}, \\ \mathbf{A} & \text{is an } n_c \times n_p \text{ matrix } (n_c = \text{number of constraints}), \\ \mathbf{b} & \text{is an } n_c\text{-dimensional vector}. \end{cases}
\end{aligned}
\tag{6.9}
$$

The expression $\mathbf{A}\mathbf{p} \leq \mathbf{b}$ specifies $n_c$ constraints: $\mathbf{p}^{\mathrm{T}}a^{(i)} \leq b^{(i)}$ for each $i$ from 1 to $n_c$, where $a^{(i)}$ is the vector from the i$^{\text{th}}$ row of $\mathbf{A}$ and $b^{(i)}$ is the corresponding element in $\mathbf{b}$.

Setting the QP parameters as described can yield the objective for a hard margin linear SVM classifier:

- $n_p = n + 1$, where $n$ is the number of features (the +1 accounts for the bias term).
- $n_c = m$, where $m$ is the number of training instances.
- $\mathbf{H}$ is an $n_p \times n_p$ identity matrix, with a zero in the top-left cell to exclude the bias term.
- $\mathbf{f} = \mathbf{0}$, an $n_p$-dimensional zero vector.
- $\mathbf{b} = -\mathbf{1}$, an $n_c$-dimensional vector filled with $-1$s.
- $\mathbf{a}^{(i)} = -t^{(i)}\dot{\mathbf{x}}^{(i)}$, where $\dot{\mathbf{x}}^{(i)}$ is $x^{(i)}$ augmented with a bias feature $\dot{\mathbf{x}}_0 = 1$.

Training a hard margin linear SVM classifier can thus be achieved by using a standard QP solver with these settings. The resultant vector $\mathbf{p}$ will include the bias term $b = p_0$ and the feature weights $w_i = p_i$ for $i = 1, 2, \ldots, n$. A similar approach can be used to solve the soft margin problem, as discussed in further exercises at the end of this chapter.

**Example 6.1 Support Vector Machine Classification**

This example applies a linear Support Vector Machine (SVM) classifier to distinguish Iris-Virginica flowers based on petal length and width. The decision boundary is derived after scaling the data, and margins are visualized. Misclassified instances within the margin are identified and marked. The model's performance is assessed using a confusion matrix and F1 score.

## 6.2 Nonlinear SVM Classification

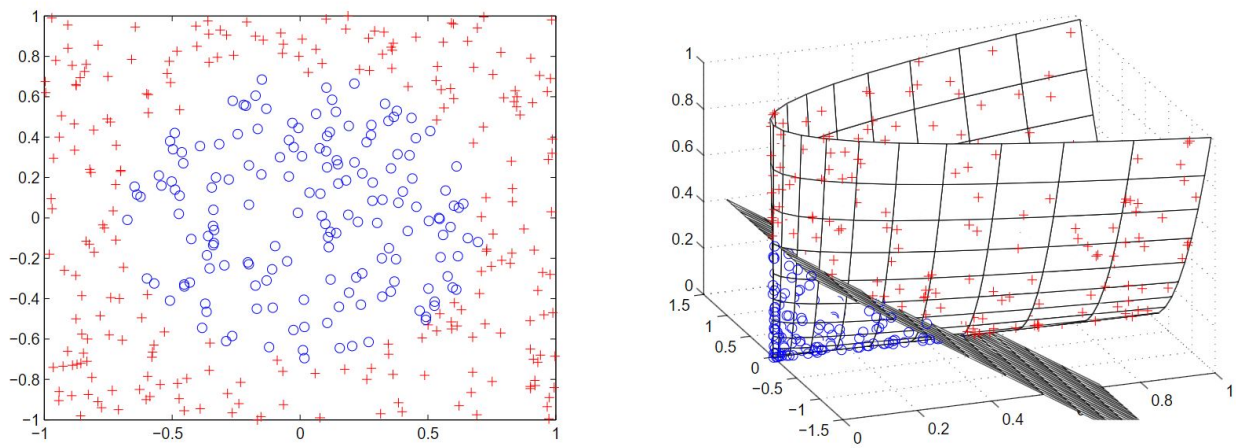


Figure 6.7: Nonlinear SVM example illustration [a]

While linear SVM classifiers are quite effective and perform exceptionally well in various scenarios, many datasets are far from being linearly separable. One strategy to address non-linear datasets is to introduce additional features, such as polynomial feature. Adding features can sometimes transform the dataset into one that is linearly separable. A representation of this technique is shown in 6.7.

A simple example of converting non-linearly separable variables is shown in figure 6.8 where the left plot displays a simple dataset with a single feature $x_1$. Clearly, this dataset is not linearly separable. However, by adding another feature $x_2 = (x_1)^2$, the dataset becomes perfectly linearly separable in two dimensions.

[a]Machine Learner, CC BY-SA 4.0 <https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons

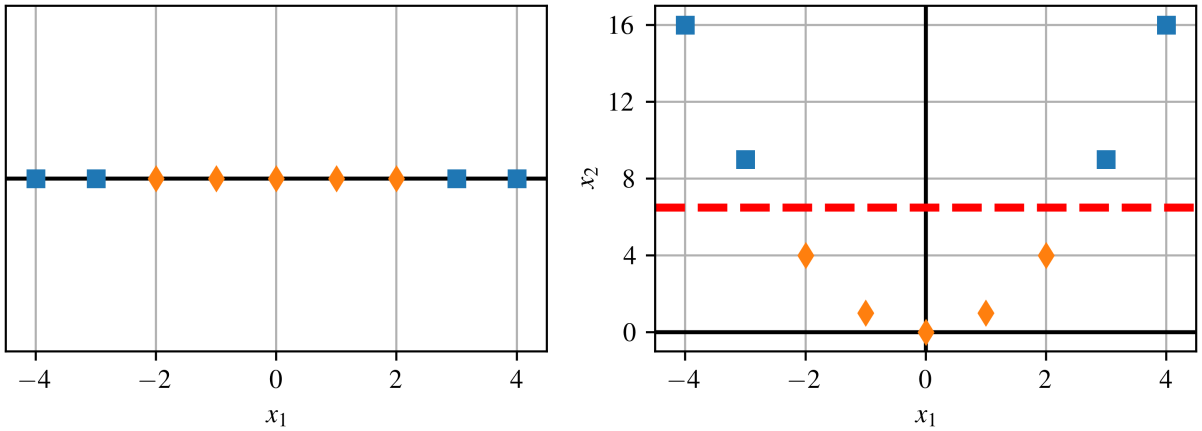


Figure 6.8: Illustration of SVM in higher dimensions

This concept can be easily implemented using Scikit-Learn by assembling a Pipeline that includes a PolynomialFeatures transformer (previously discussed in the "Polynomial Regression" section), followed by a StandardScaler and a LinearSVC. This setup can be effectively demonstrated on the moons dataset, a toy dataset for binary classification where the data points form two interleaving half circles, as shown in Figure 6.9. This dataset can be generated using the `make_moons()` function.

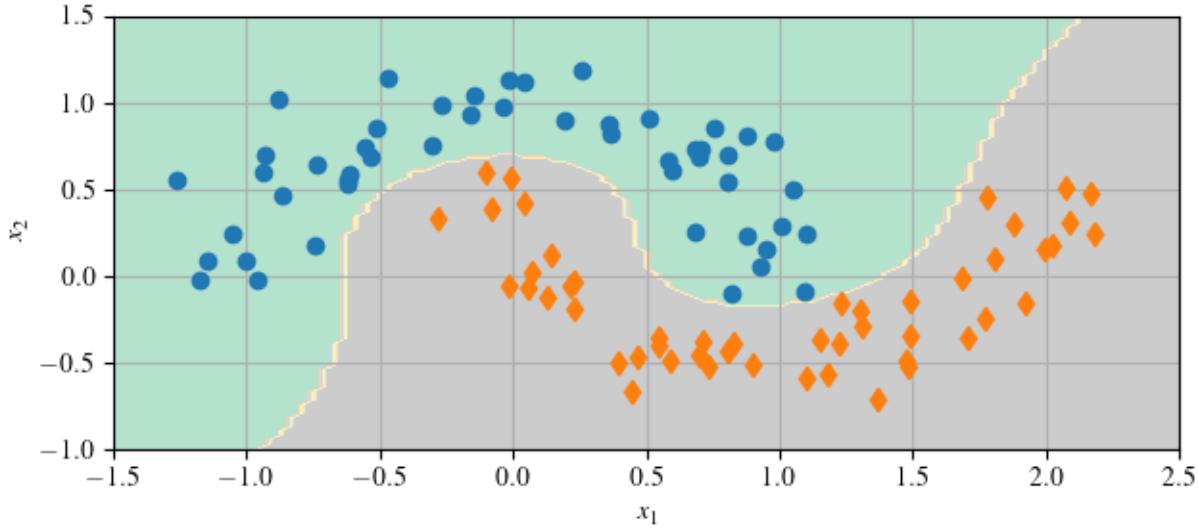


Figure 6.9: Demonstration of a SVM classifier with polynomial features.

**Example 6.2 Nonlinear Classification**
This example demonstrates nonlinear classification using the `make_moons` dataset. A poly-

nomial feature transformation is combined with a linear SVM to classify the data, and the resulting decision boundaries are visualized.

### 6.2.1 Polynomial Kernel

While adding polynomial features is straightforward and enhances the performance of various Machine Learning algorithms (not limited to SVMs), it presents limitations. Specifically, lower-degree polynomials may not adequately handle complex datasets, and higher-degree polynomials significantly increase the feature count, slowing down the model.

SVMs offer a unique solution through a remarkable mathematical technique known as the kernel trick, which allows for the benefits of high-degree polynomial features without actually expanding the feature space, thereby avoiding a rapid increase in computation. This kernel trick is incorporated within the SVC class. An application of this is demonstrated on the moons dataset:

The accompanying code configures an SVM classifier using a 3rd-degree polynomial kernel, depicted on the left of Figure 6.10. Another SVM classifier, utilizing a 10th-degree polynomial kernel, is shown on the right. Adjusting the polynomial degree can help manage model fit: reducing the degree may prevent overfitting, whereas increasing it may be necessary for underfitting scenarios. The ‘coef0‘ hyperparameter is crucial as it determines the influence of high versus low-degree polynomials in the model.

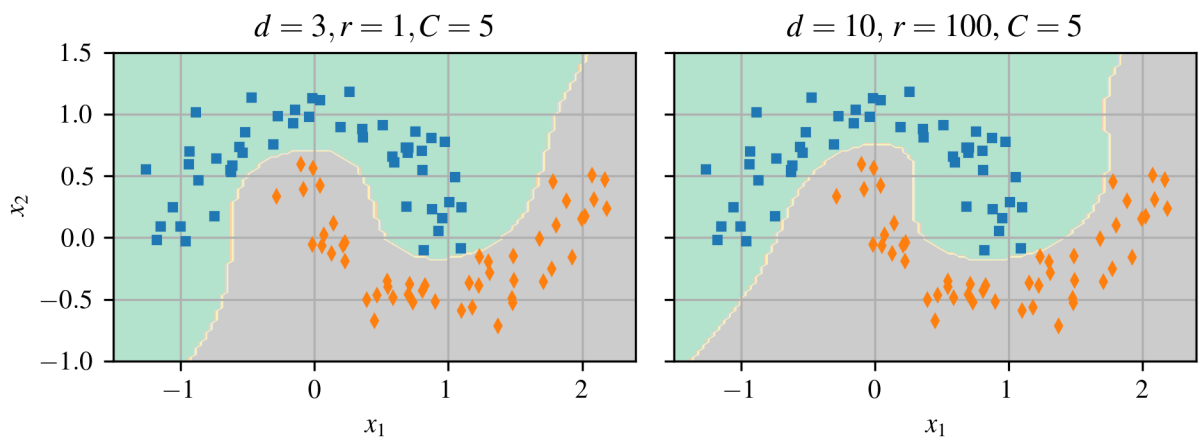


Figure 6.10: SVM polynomial kernel

NOTE

A typical method for determining optimal hyperparameter settings involves utilizing grid search techniques. Starting with a broad, coarse grid search to quickly narrow down potential candidates, followed by a more detailed, finer grid search centered on these promising values often yields faster results. Additionally, understanding the function and influence of each hyperparameter aids in efficiently targeting the most relevant areas of the hyperparameter space.

**Example 6.3 Kernel Trick with Polynomial Kernel**
This example uses the kernel trick to enable an SVM to classify non-linearly separable data. A third-degree polynomial kernel is applied to the moons dataset using Scikit-Learn's `Pipeline` and `SVC` tools, producing a curved decision boundary that cleanly separates the classes.

## 6.3 Computational Complexity

The `LinearSVC` class utilizes the `liblinear` library, optimized for linear SVMs. It does not accommodate the kernel trick but scales almost linearly with the number of training instances and features, with a training time complexity roughly proportional to $O(m \times n)$.

The precision of the algorithm can be adjusted using the tolerance hyperparameter $\varepsilon$ (referred to as `tol` in Scikit-Learn), with the default setting generally suitable for most classification tasks.

Conversely, the `SVC` class employs the `libsvm` library, supporting the kernel trick. Its training time complexity ranges between $O(m^2 \times n)$ and $O(m^3 \times n)$, making it significantly slower for large datasets, such as those with hundreds of thousands of instances. This class is best suited for complex but smaller or medium-sized datasets. It also scales efficiently with the number of features, particularly with sparse features where each instance has only a few nonzero features, roughly scaling with the average number of nonzero features per instance. The capabilities of Scikit-Learn's SVM classification classes are summarized in Table 1.

Table 1: Comparison of Scikit-Learn classes for SVM classification

| Class | Time complexity | Out-of-core support | Scaling required | Kernel trick |
|---|---|---|---|---|
| LinearSVC | $O(m \times n)$ | No | Yes | No |
| SGDClassifier | $O(m \times n)$ | Yes | Yes | No |
| SVC | $O(m^2 \times n)$ to $O(m^3 \times n)$ | No | Yes | Yes |

## 6.4 SVM Regression

The SVM algorithm showcases remarkable versatility, not only in its capability to handle both linear and nonlinear classification but also in its support for linear and nonlinear regression. This is achieved by inverting the original objective: rather than striving to create the widest possible "street" between two classes with minimal margin violations, SVM Regression aims to fit as many instances as possible within this street, while still controlling margin violations. The width of this street is adjustable via the hyperparameter $\varepsilon$. Figure 6.11 demonstrates two linear SVM Regression models trained on random linear data; one features a wide margin ($\varepsilon = 1.5$), and the other a narrower margin ($\varepsilon = 0.5$).

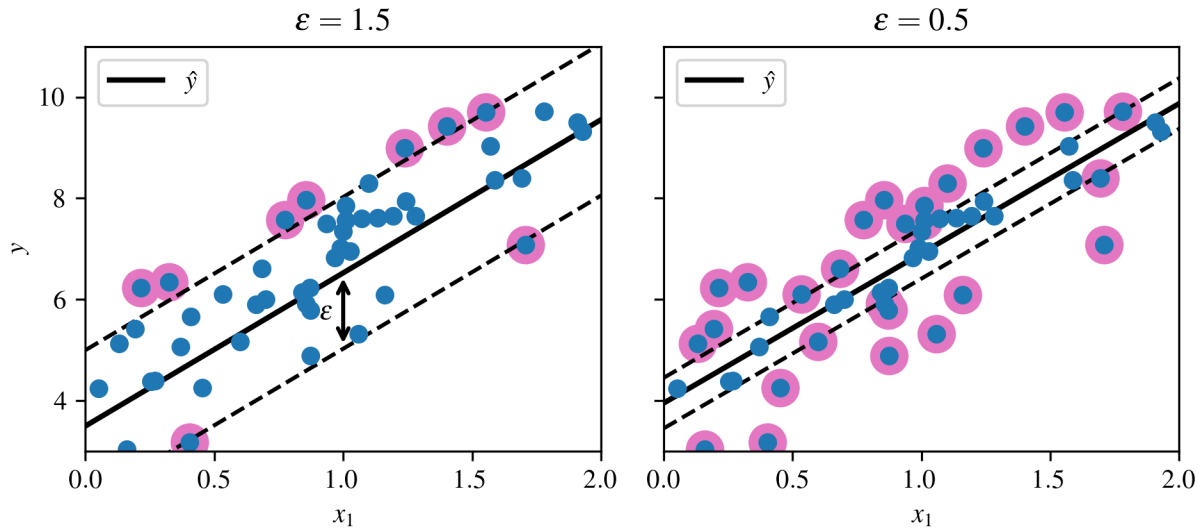


Figure 6.11: SVM Regression models with different $\varepsilon$ values.

The presence of additional training instances within the margin does not influence the predictions of the model, rendering it $\varepsilon$-insensitive. For linear SVM Regression tasks, the LinearSVR class from Scikit-Learn can be utilized. When addressing nonlinear regression challenges, a kernelized SVM model can be employed. For instance, Figure 6.12 illustrates SVM Regression applied to a randomly generated quadratic dataset using a 2nd-degree polynomial kernel, with varying degrees of regularization depicted in the plots.

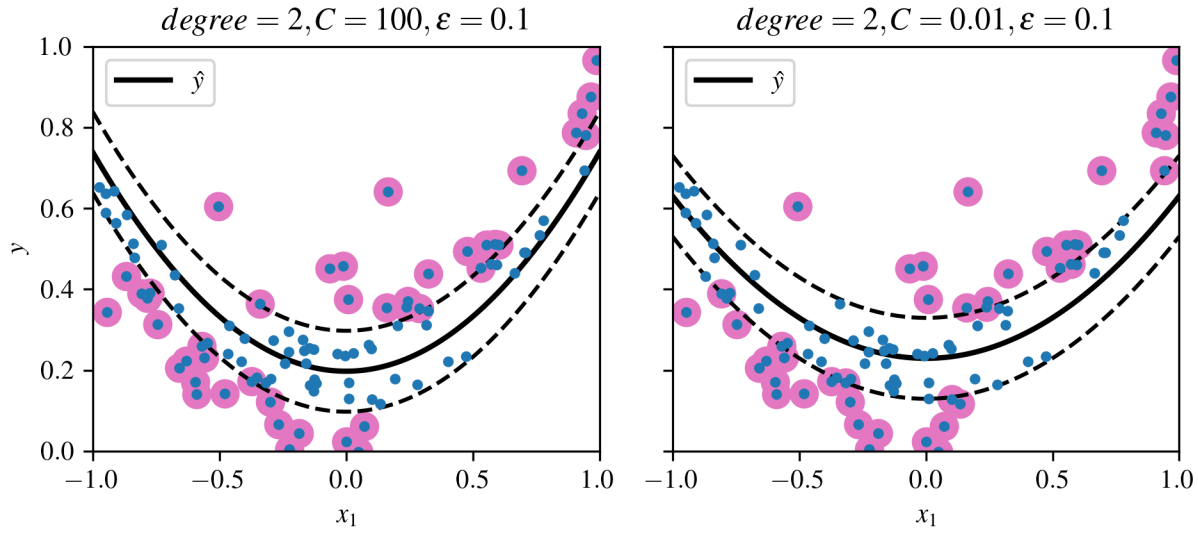


Figure 6.12: SVM regression with a 2nd-degree polynomial kernel, showcasing different regularization levels.

The SVR class, supporting the kernel trick and acting as the regression counterpart to the SVC class, performs well with small to medium-sized datasets but slows considerably as dataset size

increases. In contrast, the LinearSVR class, akin to the LinearSVC class, scales linearly with the size of the training set.